



Divide y vencerás

Simona Bernardi

Universidad de Zaragoza

Presentación adaptada de la original de J. Campos

Este documento está sujeto a una licencia de uso Creative Commons. No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.





Resumen

- 1 Introducción
- 2 Multiplicación de enteros grandes
- 3 Multiplicación de matrices
- 4 Potenciación de enteros
- 5 Otros ejemplos (búsqueda dicotómica, ordenación, búsqueda de la mediana)



Método general

Esquema algorítmico

```
1  tipoRet divideVenceras(tipoProblema P){  
2    if esPequeño(P) return resolverDirectamente(P);  
3    else {  
4      descomponer P en P_1, P_2,...,P_k (k>=1);  
5      for each  $i \in \{1, \dots, k\}$  {  
6        sol_i = divideVenceras(P_i);  
7      }  
8      return combinar(sol_1,...,sol_k);  
9    }  
10 }
```

- Si $k = 1$, el esquema anterior se llama **técnica de reducción**



Coste computacional (tiempo)

Sean:

- n el tamaño de P
- n_i el tamaño del sub-problema $P_i (1 \leq i \leq k)$

$$T(n) = \begin{cases} t_{base}(n) & n \text{ es pequeño} \\ T(n_1) + T(n_2) + \dots + T(n_k) + t(n) & \text{en otro caso} \end{cases}$$

- $t_{base}(n)$: tiempo para resolver directamente P
- $T(n_i)$: tiempo para resolver $P_i (1 \leq i \leq k)$
- $t(n)$: tiempo para descomponer P y combinar las soluciones



Análisis del coste: un ejemplo

Descomposición

Sea A un algoritmo que emplea un tiempo cuadrático para resolver un problema P de tamaño n :

$$t_A(n) \leq k_0 n^2, \quad k_0 \text{ constante}$$

Enfoque: descomposición del problema

- Supongamos que P se puede descomponer en tres sub-problemas $P_i (i = 1, 2, 3)$ de tamaño $\frac{n}{2}$
- Los P_i se pueden resolver utilizando A
- Las soluciones sol_i se pueden combinar para resolver el problema P
- El tiempo empleado para descomponer P y combinar las tres soluciones sol_i es:

$$t(n) \leq k_1 n, \quad k_1 \text{ constante}$$



Análisis del coste: un ejemplo

Descomposición

Tenemos un nuevo algoritmo B para resolver P con un tiempo:

$$\begin{aligned}t_B(n) &= 3t_A\left(\frac{n}{2}\right) + t(n) \\&\leq 3k_0\left(\frac{n}{2}\right)^2 + k_1n \\&= \frac{3}{4}k_0n^2 + k_1n\end{aligned}$$

⇒ Para n “suficientemente grande”, B es un 25 % más rápido que A



Análisis del coste: un ejemplo

Recurrencia

Repetimos la descomposición ...

Enfoque: divide y vencerás

- El nuevo algoritmo para resolver P con tiempo:

$$t(n) = \begin{cases} t_A(n) & n \leq n_0 \text{ (umbral)} \\ 3t(\frac{n}{2}) + t(n) & n > n_0 \end{cases}$$

- $t_A(n)$ es el tiempo del algoritmo que calcula directamente la solución
- n_0 es el umbral por encima del cual el algoritmo se usa de forma recursiva
- $t(n)$ es el tiempo de la descomposición y combinación



Análisis del coste: un ejemplo

iteración	n.problemas	tamaño
0	1	n
1	3	n/2
2	9	n/4
3	27	n/8
...	...	...
k	3^k	$n/2^k$

- Si el tamaño $n/2^k = 1$:

$$\Rightarrow k = \log_2 n$$

- $\Rightarrow 3^{\log_2 n}$ problemas, es decir:

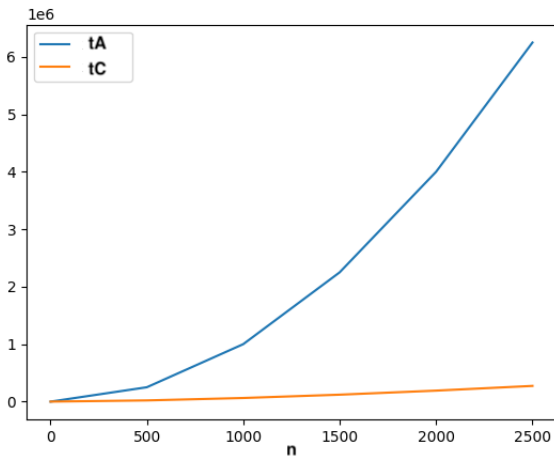
$$3^{\log_2 n} = 3^{\frac{\log_3 n}{\log_3 2}} =$$

$$(3^{\log_3 n})^{\frac{1}{\log_3 2}} = n^{\frac{1}{\log_3 2}} = n^{\log_2 3}$$

- Hay $n^{\log_2 3} \approx n^{1,6}$ problemas de tamaño 1
- Coste del algoritmo: $t_C(n) \in O(n^{\log_2 3}) \approx O(n^{1,6})$
- De momento, no tenemos en cuenta del coste $t(n)$



Análisis del coste: un ejemplo





Análisis del coste

Solución de la ecuación en recurrencias

La complejidad en tiempo de muchos algoritmos basados en el esquema divide y vencerás se describe de la forma:

$$T(n) = \begin{cases} t_{base}(n) & n \leq n_0, \text{ umbral} \\ aT(n/b) + t(n) & n > n_0 \end{cases}$$

- a y b son constantes y se conocen
- Sean $t_{base}(n)$, $t(n)$ conocidos y $n = b^k (k > 0)$

¿Cómo resolvemos la ecuación en recurrencias?

⇒ Se suele resolver con el *método de sustitución*



Análisis del coste

Master Theorem: análisis asintótico

- Dada la ecuación en recurrencias:

$$T(n) = aT(n/b) + O(n^d), \quad a \geq 1, b > 1, d \text{ constantes}$$

- Entonces:

$$T(n) = \begin{cases} O(n^d \log n) & a = b^d \\ O(n^d) & a < b^d \\ O(n^{\log_b a}) & a > b^d \end{cases}$$

- a es el número de subproblemas
- b es factor de reducción del tamaño de entrada
- Se necesita n^d de tiempo de trabajo para crear los sub-problemas y combinar sus soluciones.

Multiplicación de enteros grandes



- Las operaciones elementales de suma y multiplicación tienen un coste **constante** para implementaciones hardware (enteros “no muy grandes”)



Multiplicación de enteros grandes

- Las operaciones elementales de suma y multiplicación tienen un coste **constante** para implementaciones hardware (enteros “no muy grandes”)
- ¿El coste sigue siendo constante para enteros “muy grandes”?



Multiplicación de enteros grandes

- Las operaciones elementales de suma y multiplicación tienen un coste **constante** para implementaciones hardware (enteros “no muy grandes”)
- ¿El coste sigue siendo constante para enteros “muy grandes”?
- Consideramos enteros de n cifras
 - Suma, resta: $O(n)$ **tiempo lineal**
 - Multiplicación, división entera por potencias positivas de la base: $O(n)$ **tiempo lineal** (desplazamiento de las cifras)
 - Multiplicación con el algoritmo clásico: $O(n^2)$ **tiempo cuadrático**



Multiplicación de enteros grandes

- Las operaciones elementales de suma y multiplicación tienen un coste **constante** para implementaciones hardware (enteros “no muy grandes”)
- ¿El coste sigue siendo constante para enteros “muy grandes”?
- Consideramos enteros de n cifras
 - Suma, resta: $O(n)$ **tiempo lineal**
 - Multiplicación, división entera por potencias positivas de la base: $O(n)$ **tiempo lineal** (desplazamiento de las cifras)
 - Multiplicación con el algoritmo clásico: $O(n^2)$ **tiempo cuadrático**

¿Cómo multiplicar dos números enteros de forma más eficiente que con el algoritmo clásico?



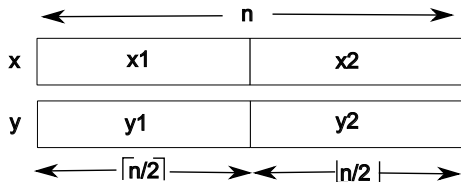
Multiplicación de enteros grandes

A. Karatsuba, Y. Ofman. Multiplication of multidigit numbers on automata. Dokl. Akad. Nauk SSSR, 145, pp. 209-294, 1962.

⇒ Técnica de divide y vencerás para la multiplicación de enteros grandes

- Sean x e y dos enteros de n cifras (ejemplo base 10)
- Se descomponen en mitades de igual tamaño s

$$\begin{cases} x = 10^s x_1 + x_2 & 0 \leq x_2 < 10^s \\ y = 10^s y_1 + y_2 & 0 \leq y_2 < 10^s \end{cases}$$



El producto es:

$$xy = x_1 y_1 10^{2s} + (x_1 y_2 + x_2 y_1) 10^s + x_2 y_2$$



Multiplicación de enteros grandes

Primer intento ...

```
1  // x e y son dos enteros grandes
2  // Devuelve el resultado de multiplicar x por y
3  tipoInt mult(tipoInt x, tipoInt y){
4      n = máx(nCifras(x), nCifras(y));
5      if esPequeño(n) return x*y; // caso base
6      else{
7          s = n/2; // división entera
8          x1 = x/10s; x2 = x % 10s; // divisiones enteras y op. módulo
9          y1 = y/10s; y2 = y % 10s;
10         return mult(x1,y1)*102s +
11                ( multi(x1,y2) + mult(x2,y1) )*10s +
12                mult(x2, y2);
13     }
14 }
```

¿Cuál es el coste temporal de mult?



Multiplicación de enteros grandes

Coste temporal de `mult`

- Las sumas, las multiplicaciones y divisiones por potencias de 10: tiempo lineal
- El módulo una potencia de 10: tiempo lineal
- Ecuación en recurrencias (n potencia de 2):

$$T(n) = 4T(n/2) + k_2n, \quad k_2 \text{ constante}$$

- Podemos aplicar el Master Theorem para evaluar el coste asintótico de `mult`

$$T(n) \in O(n^2)$$

¿¡Y no hemos ganado nada!?



Multiplicación de enteros grandes

Segundo intento: algoritmo de Karatsuba y Ofman

¿Podemos reducir el número de llamadas recursivas en el algoritmo? Sí ☺

$$\underbrace{(x_1 + x_2)(y_1 + y_2)}_r = \underbrace{x_1 y_1}_p + (x_1 y_2 + x_2 y_1) + \underbrace{x_2 y_2}_q$$

```

1  // x e y son dos enteros grandes.
2  // Devuelve el resultado de multiplicar x por y
3  tipoInt multV2(tipoInt x, tipoInt y){
4      n = máximo(nCifras(x), nCifras(y));
5      if esPequeño(n) return x*y; // caso base
6      else{ s = n/2; // división entera
7          x1 = x/10s; x2 = x % 10s; // divisiones enteras y op. módulo
8          y1 = y/10s; y2 = y % 10s;
9          r = multV2(x1+x2, y1+y2);
10         p = multV2(x1, y1); q = multV2(x2, y2);
11         return p*102s + (r-p-q)*10s + q;
12     }
13 }
```



Multiplicación de enteros grandes

Coste temporal de `multV2`

- Ecuación en recurrencias:

$$T(n) \leq 3T(n/2) + k_2n, \quad n \text{ grande, } k_2 \text{ constante}$$

- Aplicamos el Master Theorem, en la fórmula simplificada:

$$T(n) \in O(n^{\log_2 3}) \approx O(n^{1,6})$$

⇒ Una buena implementación no debe usar la base 10 sino la base más grande posible que permita multiplicar dos “cifras” (de esa base) directamente en hardware.

Multiplicación de enteros grandes

¿Se puede mejorar la cota superior del coste?

- Toom-Cook (1963)
 - Divide en 5 problemas de tamaño $n/3$ (DyV)
 - Coste asintótico $O(n^{1,465})$





Multiplicación de enteros grandes

¿Se puede mejorar la cota superior del coste?

- Toom-Cook (1963)
 - Divide en 5 problemas de tamaño $n/3$ (DyV)
 - Coste asintótico $O(n^{1.465})$
- Schönhage-Strassen (1971):
 - Convierte el problema en la multiplicación de polinomios y aplica la transformada rápida de Fourier para multiplicarlos (también DyV)
 - Coste asintótico $O(n \log n \log(\log n))$
 - El algoritmo más rápido hasta el 2007, en particular supera los anteriores con números de $[10,000; 100,000]$ cifras decimales
- ...



Multiplicación de enteros grandes

¿Se puede mejorar la cota superior del coste?

- Toom-Cook (1963)
 - Divide en 5 problemas de tamaño $n/3$ (DyV)
 - Coste asintótico $O(n^{1.465})$
- Schönhage-Strassen (1971):
 - Convierte el problema en la multiplicación de polinomios y aplica la transformada rápida de Fourier para multiplicarlos (también DyV)
 - Coste asintótico $O(n \log n \log(\log n))$
 - El algoritmo más rápido hasta el 2007, en particular supera los anteriores con números de $[10,000; 100,000]$ cifras decimales
- ...
- Harvey-van der Hoeven (2021)
 - Coste $O(n \log n) \Rightarrow$ “Algoritmo galáctico” ☺
 - Problema: constantes multiplicativas que lo hacen muy lento en la práctica

Multiplicación de enteros grandes

Ejemplo de reducción

- Dado el problema de multiplicación de dos enteros grandes
- ⇒ Lo transformo en un problema de multiplicación de dos polinomios

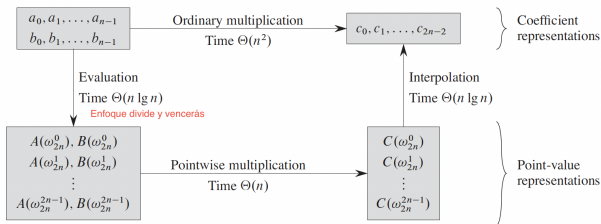




Multiplicación de enteros grandes

Ejemplo de reducción

- Dado el problema de multiplicación de dos enteros grandes
- ⇒ Lo transformo en un problema de multiplicación de dos polinomios
- Aplico un algoritmo eficiente al problema de multiplicación de polinomios (transformación rápida de Fourier)



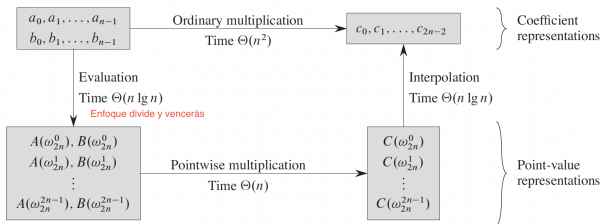
Cormen et al. *Introduction to algorithms*.



Multiplicación de enteros grandes

Ejemplo de reducción

- Dado el problema de multiplicación de dos enteros grandes
- ⇒ Lo transformo en un problema de multiplicación de dos polinomios
- Aplico un algoritmo eficiente al problema de multiplicación de polinomios (transformación rápida de Fourier)



Cormen et al. *Introduction to algorithms*.

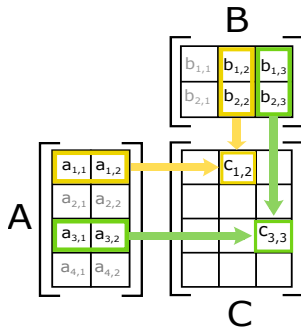
- Transformo la solución (polinomio producto) en la solución al problema inicial



Multiplicación de matrices

- Dadas las dos matrices A y B de dimensión $n \times n$
- La matriz producto $C = AB$:

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}, \quad i, j = 1, \dots, n$$



El algoritmo clásico calcula C en un tiempo $\Theta(n^3)$



Multiplicación de matrices

Estrategia divide y vencerás

- Asumimos $n = 2^k$ por simplicidad
- Dividimos las dos matrices A y B en 4 sub-matrices de dimensión $\frac{n}{2} \times \frac{n}{2}$
- Calculamos $C = AB$ considerando las sub-matrices:

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

- Donde:

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$



Multiplicación de matrices

Estrategia divide y vencerás

- Si $n = 2$, se reduce a 8 multiplicaciones y 4 sumas
- Si $n > 2$, son 8 productos y 4 sumas de matrices de dimensión $\frac{n}{2} \times \frac{n}{2}$
 - Se pueden calcular de forma recursiva considerando matrices de tamaño más pequeño ...
- Coste en tiempo:

$$T(n) = \begin{cases} k_1 & n = 2, k_1 \text{ constante} \\ 8T(n/2) + k_2 n^2 & n > 2, k_2 \text{ constante} \end{cases}$$

donde, la suma de matrices tiene un coste $\Theta(n^2)$

No hay mejora con respecto al algoritmo clásico: $T(n) \in O(n^3)$.

¿Se puede mejorar? ☺



Multiplicación de matrices

V. Strassen, *Gaussian elimination is not optimal*. Numerische Mathematik, 13, pp. 354-356, 1969.

Idea de Strassen

Reducir el número de multiplicaciones a costa de aumentar el número de sumas y restas, e introducir variables auxiliares

- Strassen calcula primero 7 matrices $\frac{n}{2} \times \frac{n}{2}$:

$P = (A_{11} + A_{22})(B_{11} + B_{22})$	$Q = (A_{21} + A_{22})B_{11}$
$R = A_{11}(B_{12} - B_{22})$	$S = A_{22}(B_{21} - B_{22})$
$T = (A_{11} + A_{12})B_{22}$	$U = (A_{21} - A_{11})(B_{11} + B_{12})$
$V = (A_{12} - A_{22})(B_{21} + B_{22})$	

- Después usa las siguientes formulas para calcular los C_{ij} :

$$C = \begin{bmatrix} P + S - T + V & R + T \\ Q + S & P + R - Q + U \end{bmatrix}$$



Multiplicación de matrices

Coste temporal del método de Strassen

- 7 multiplicaciones, 18 sumas y restas.
- El número de sumas y restas puede reducirse utilizando más variables auxiliares para evitar cálculos repetidos.
- Coste en tiempo:

$$T(n) = \begin{cases} k_1 & n = 2, \text{ } k_1 \text{ constante} \\ 7T(n/2) + k_2 n^2 & n > 2, \text{ } k_2 \text{ constante} \end{cases}$$

- Se pueden multiplicar matrices $n \times n$ en tiempo $O(n^{\log 7}) \approx O(n^{2,81})$
- Si n no es potencia de 2, siempre se pueden añadir filas y columnas nulas hasta que lo sea



Multiplicación de matrices

Método de Strassen comparado con el algoritmo clásico

Eficiencia en la práctica	mejor si $n > 100$ y matrices densas (estudio empírico)
Estabilidad	menos estable (para errores similares en los datos produce mayores errores en los resultados)
Implementación	más complejo
¿Es paralelizable?	más difícil
Coste en espacio	espacio adicional mayor

Multiplicación de matrices

Consideraciones finales



- El método de Strassen dio también ideas para otras operaciones sobre matrices (inversión, determinantes, ...)



Multiplicación de matrices

Consideraciones finales

- El método de Strassen dio también ideas para otras operaciones sobre matrices (inversión, determinantes, ...)
- Se conocen algoritmos teóricamente mejores: la actual cota superior está en $O(n^{2,371552})$
 - V.V. Williams et al. *New Bounds for Matrix Multiplication: from Alpha to Omega*, April 2024. <https://arxiv.org/abs/2307.07970>



Multiplicación de matrices

Consideraciones finales

- El método de Strassen dio también ideas para otras operaciones sobre matrices (inversión, determinantes, ...)
- Se conocen algoritmos teóricamente mejores: la actual cota superior está en $O(n^{2,371552})$
 - V.V. Williams et al. *New Bounds for Matrix Multiplication: from Alpha to Omega*, April 2024. <https://arxiv.org/abs/2307.07970>
- Avances prácticos en la multiplicación de matrices con algoritmos de inteligencia artificial
 - Fawzi, A. et al. *Discovering faster matrix multiplication algorithms with reinforcement learning*, Nature 610, 47–53 (2022).
<https://doi.org/10.1038/s41586-022-05172-4>



Potenciación de enteros

Problema: dados dos enteros positivos a y n , calcular a^n

Una solución

```
1  // a y n son dos enteros positivos.
2  // Devuelve  $a^n$ .
3  tipoInt potenciación(tipoInt a, tipoInt n){
4      r = 1;
5      for each  $i \in \{1, \dots, n\}$  { r=r*a; }
6      return r;
7  }
```

- Si $r = r*a$ se considera “operación elemental”, el coste temporal está en $\Theta(n)$.
- Sin embargo incluso valores pequeños de a y n hacen que este algoritmo produzca desbordamiento de enteros (por ejemplo, 15^{17} no cabe en un entero de 64 bits).



Potenciación de enteros

Coste temporal de potenciación

- Sean

- r_i el resultado de la multiplicación $r * a$ en el paso i
- $m_i = n_cifras(r_i)$, $m_1 = n_cifras(a) = m$

- Entonces

$$m + m_i - 1 \leq n_cifras(r_{i+1}) \leq m + m_i \quad \text{por inducción} \implies$$

$$im - i + 1 \leq n_cifras(r_{i+1}) \leq im$$

- Coste temporal del algoritmo

$$\sum_{i=1}^{n-1} M(m, im - i + 1) \leq T(m, n) \leq \sum_{i=1}^{n-1} M(m, im)$$

donde $M(p, q)$ es el coste de multiplicar dos enteros de tamaño p y q



Potenciación de enteros

Coste temporal de potenciación prohibitivo para enteros grandes

- Si utilizamos el algoritmo clásico de multiplicación $M(p, q) \in \Theta(pq)$
- Por tanto

$$T(m, n) \leq \sum_{i=1}^{n-1} M(m, im) \leq \sum_{i=1}^{n-1} k_1 mim = k_1 m^2 \sum_{i=1}^{n-1} i \leq k_1 m^2 n^2$$

- Es decir $T(m, n) \in O(m^2 n^2)$
 - Se puede demostrar que $T(m, n) \in \Theta(m^2 n^2)$
- Si se usa el algoritmo `multV2` (de Karatsuba y Ofman) para multiplicar $T(m, n) \in \Theta(m^{\log_2 3} n^2)$



Potenciación de enteros

Otra solución

- Observamos que: $a^n = (a^{\frac{n}{2}})^2$
- ¿Si n es impar?

Ejemplo $n = 25$

Vamos a reescribir:

$$a^{25} = \left(((a^2 a)^2)^2 \right)^2 a$$

- ¿Cuántas veces se multiplica? ¿Y cuántas veces se eleva al cuadrado?
- Descomponemos el problema inicial:

$$a^{25} = a^{24} a; \quad a^{24} = (a^{12})^2; \quad \dots$$

Enfoque divide y vencerás



Potenciación de enteros

Otra solución

```
1  // a y n son dos enteros positivos.
2  // Devuelve a^n.
3  tipoInt potenciaciónV2(tipoInt a, tipoInt n){
4      if n=0 return 1;
5      else{
6          if esImpar(n){
7              r = potenciaciónV2(a,n-1);
8              return a*r;
9          }
10         else{
11             r = potenciaciónV2(a,n/2);
12             return r*r;
13         }
14     }
```

- Es un algoritmo de reducción



Potenciación de enteros

Coste temporal de potenciaciónV2

- Tiempo $O(h(n) \times M(z))$
 - $h(n)$ número de multiplicaciones
 - $M(z)$ cota del tiempo para multiplicar dos enteros menores de z
- Tenemos:

$$h(n) = \begin{cases} 0 & n = 0 \\ 1 + h(n-1) & n \text{ es impar} \\ 1 + h(n/2) & \text{en otro caso} \end{cases}$$

- Por tanto $h(n)$ es logarítmico en n .
- Teniendo en cuenta el número de multiplicaciones necesarias, potenciaciónV2 es más eficiente que potenciación.
- ¿Es significativamente mejor cuando se tiene en cuenta el tiempo invertido en esas multiplicaciones?



Potenciación de enteros

- El algoritmo potenciaciónV2 es razonablemente bueno, pero no es óptimo
- Su eficiencia depende del algoritmo que se utilice para multiplicar enteros grandes

	multiplicación	
	clásica	DyV
potenciación	$\Theta(m^2 n^2)$	$\Theta(m^{\log_2 3} n^2)$
potenciaciónV2	$\Theta(m^2 n^2)$	$\Theta(m^{\log_2 3} n^{\log_2 3})$



Búsqueda

Sean $v_i (i=1, \dots, n)$ n elementos ordenados de forma no decreciente, dado un x determinar si x pertenece a la lista. En caso positivo, devolver el index j tal que $v_j = x$. En caso negativo, devolver cero.

Una solución con coste $\Theta(n)$ (casos promedio y peor)

```

1  // Tupla v de n el. ordenados (forma no decreciente). x el. buscado.
2  //  $\exists j \in [1, n] : v[j] = x \Rightarrow \text{secuencial}(v, n, x) = j$ , otro caso  $\text{secuencial}(v, n, x) = 0$ .
3  int secuencial(tipoElemento v[], int n, tipoElemento x){
4      j = 1; encontrado = false;
5      while j<=n and not encontrado {
6          if v[j] == x { encontrado = true; }
7          else j=j+1;
8      }
9      if encontrado return j;
10     else return 0;
11 }
```



Búsqueda dicotómica

Enfoque divide y vencerás: coste $\Theta(\log n)$ (casos promedio y peor)

```

1 // Tupla v de n elementos ordenados (forma no decreciente).
2 // x elemento buscado.
3 //  $\exists j \in [1, n] : v[j] = x \Rightarrow \text{dicot}(v, 1, n, x) = j$ , otro caso  $\text{dicot}(v, 1, n, x) = 0$ .
4
5 int dicot(tipoElemento v[], int inf, int sup, tipoElemento x){
6     if inf == sup { // Caso problema pequeño
7         if x == v[inf] return inf;
8         else return 0;
9     }
10    else { // Descomponemos el problemas en un sub-problema más pequeño
11        medio =  $\lfloor (inf + sup) / 2 \rfloor$ ;
12        if x == v[medio] return medio;
13        else{
14            if x < v[medio] return dicot(v, inf, medio-1, x);
15            else return dicot(v, medio+1, sup, x);
16        }
17    }
18 }
```



Ordenación por fusión

Enfoque divide y vencerás

Sean $v_i (i=1, \dots, n)$ n elementos dotados de una relación de orden total, queremos ordenarlos en orden no decreciente.

```
1 // Tupla v de n elementos
2 // fusión ordena la tupla v de forma no decreciente
3 void fusión(tipoElemento v[], int inf, int sup){
4     if inf < sup {
5         medio = ⌊ (inf + sup) / 2 ⌋; // Descomponer el problema
6         // Resolver los dos sub-problemas
7         fusión(v, inf, medio);
8         fusión(v, medio+1, sup);
9         combina(v, inf, medio, sup); // Combinar las dos soluciones
10    }
11 }
```



Ordenación por fusión

Combinación de las dos soluciones

```

1  // Las tuplas  $v[inf, \dots, medio]$  y  $v[medio+1, \dots, sup]$  están ordenadas ( $\leq$ ).
2  // combina las dos tuplas para obtener la tupla  $v[inf, \dots, sup]$  ordenada ( $\leq$ ).
3  void combina(tipoElemento v[], int inf, int medio, int sup){
4      h = inf; idx = 1; j = medio+1;
5      while h <= medio and j <= sup {
6          if v[h] <= v[j] { b[idx] = v[h]; h = h + 1; } // b tupla auxiliar
7          else { b[idx] = v[j]; j = j + 1; }
8          idx = idx + 1;
9      }
10     if h > medio {
11         for each  $k \in \{j, \dots, sup\}$  { b[idx] = v[k]; idx = idx + 1; }
12     }
13     else {
14         for each  $k \in \{h, \dots, medio\}$  { b[idx] = v[k]; idx = idx + 1; }
15     }
16     for each  $k \in \{inf, \dots, sup\}$  { v[k] = b[k-inf+1]; }
17 }

```



Ordenación por fusión

Coste computacional

Ecuación en recurrencias

$$T(n) = \begin{cases} k_1 & n = 1, k_1 \text{ constante} \\ 2T(n/2) + k_2 n & n > 1, k_2 \text{ constante} \end{cases}$$

- La descomposición precisa tiempo constante: $\Theta(1)$
- La fusión precisa tiempo lineal: $\Theta(n)$
- Utilizando el Master Theorem (pág. 12): $T(n) \in O(n \log n)$
- Este algoritmo es especialmente adecuado para listas y ficheros
- Variante: el problema pequeño se resuelve con el algoritmo de inserción



Ordenación rápida

C.A.R. Hoare, *Quicksort*, Computer Journal, 5(1), pp. 10-15, 1962.

```
1 // Tupla v de n elementos
2 // quicksort ordena la tupla v de forma no decreciente
3
4 void quicksort(tipoElemento v[], int inf, int sup){
5     if inf < sup {
6         idxPivote = divide(v, inf, sup); // Descomponer el problema
7         // Resolver los dos sub-problemas
8         quicksort(v, inf, idxPivote-1);
9         quicksort(v, idxPivote+1,sup);
10        // No hace falta combinar las soluciones :)
11    }
12 }
```




Ordenación rápida

Partición

```

1  // Dada la tupla  $v[inf, \dots, sup]$  y el pivote inicial  $piv = v[inf]$ 
2  // divide crea una partición de  $v$ :  $v_A[inf, idx)$ ,  $v_B(idx, sup]$ 
3  // de forma que  $piv = v[idx]$   $inf \leq idx \leq sup$ , y  $\forall x \in v_A : x \leq piv$  y  $\forall x \in v_B : x > piv$ 
4  // y devuelve  $idx$ 
5
6  int divide(tipoElemento v[], int inf, int sup){
7      piv = v[inf]; i=inf+1; j=sup;
8      while i <= j {
9          if v[i] <= piv { i=i+1; }
10         else if v[j] > piv { j=j-1; }
11         else { intercambiar(v[i],v[j]); }
12     }
13     intercambiar(v[inf],v[j]);
14     return j;
15 }

```

- Reordena el vector *in situ* y devuelve el índice del pivote



Ordenación rápida

Coste computacional

- El tiempo de ejecución depende de si la partición es equilibrada o no

Caso peor: partición no equilibrada

- En cada llamada recursiva $|v_A| = 1$, $|v_B| = n - 1$ (vector ordenado!)
- Coste de la partición $\Theta(n)$ ($\Rightarrow$ **ejercicio**)
- Ecuación en recurrencias:

$$T(n) \in \begin{cases} k_1 & n = 1, k_1 \text{ constante} \\ T(n-1) + k_2 n & n > 1, k_2 \text{ constante} \end{cases}$$

- Aplicando el método de sustitución obtenemos:

$$T(n) \in \sum_{k=1}^n \Theta(k) = \Theta\left(\sum_{k=1}^n k\right) = \Theta(n^2)$$



Algoritmo de ordenación rápida – Análisis del coste Θ

- El tiempo de ejecución depende de si la partición es equilibrada o no

Caso mejor: partición equilibrada

- En cada llamada recursiva $|v_A| = \lfloor \frac{n}{2} \rfloor$, $|v_B| = \lceil \frac{n}{2} \rceil - 1$
- Coste de la partición $\Theta(n)$
- Ecuación en recurrencias:

$$T(n) \in \begin{cases} k_1 & n = 1, k_1 \text{ constante} \\ 2T(n/2) + k_2 n & n > 1, k_2 \text{ constante} \end{cases}$$

$\Rightarrow$ Coste $T(n) \in O(n \log n)$

- El coste promedio es similar al caso mejor.



Selección y búsqueda de la mediana

También conocida como “cálculo de los estadísticos de orden i -ésimo”

Problema general

Dado un conjunto A de n elementos (distintos) y un número i ($1 \leq i \leq n$), encontrar el elemento $x \in A$ mayor de “exactamente” $i - 1$ elementos.

- Es decir, x es el elemento en posición i si A estuviese ordenado crecientemente.
- Casos particulares:
 - El mínimo: $i = 1$
 - El máximo: $i = n$
 - ¿La **mediana**?
 - Si n es impar, hay una única mediana en posición $(n + 1)/2$
 - Si n es par, hay dos medianas en posiciones $\lfloor (n + 1)/2 \rfloor$ y $\lceil (n + 1)/2 \rceil$



Selección y búsqueda de la mediana

Soluciones

- Consideramos el problema de encontrar la mediana de posición $\lfloor (n+1)/2 \rfloor$
- Se puede resolver en tiempo $O(n \log n)$:
 - 1 Ordenar A con un algoritmo como mergesort, heapsort, ...
 - 2 Extraer el elemento en la posición $i = \lfloor (n+1)/2 \rfloor$

Hay otra solución inspirada en el algoritmo de ordenación rápida (pero ahora sólo resuelve uno de los dos sub-problemas)



Selección y búsqueda de la mediana

```
1  // Dados una tupla v[inf,..,sup] y  $inf \leq i \leq sup$ 
2  // devuelve el i-ésimo elemento más pequeño.
3  tipoElemento selección(tipoElemento v[], int inf, int sup, int i){
4      if inf == sup { return v[inf]; }
5      else {
6          idx = divide(v, inf, sup); // Descomponer el problema
7          k = idx - inf + 1; // N. elementos del sub-vector v[inf,..,idx]
8          if i == k { return v[idx]; }
9          else if i < k { return selección(v, inf, idx-1,i); }
10         else { return selección(v, idx+1,sup,i-k); }
11     }
12 }
```

- divide encuentra la partición de la tupla y devuelve el índice del pivote (algoritmo de ordenación rápida)
- selección(v,1,n,(n+1)/2) devuelve la mediana



Selección y búsqueda de la mediana

Coste temporal

- Como en el método de ordenación rápida, una selección desafortunada del pivote conduce en el **caso peor** a un tiempo **cuadrático**
- No obstante, el coste promedio es **lineal**
- Existe una mejora que permite un coste lineal también en el peor caso, aunque en la práctica, en muchos casos, se comporta mejor el algoritmo anterior.